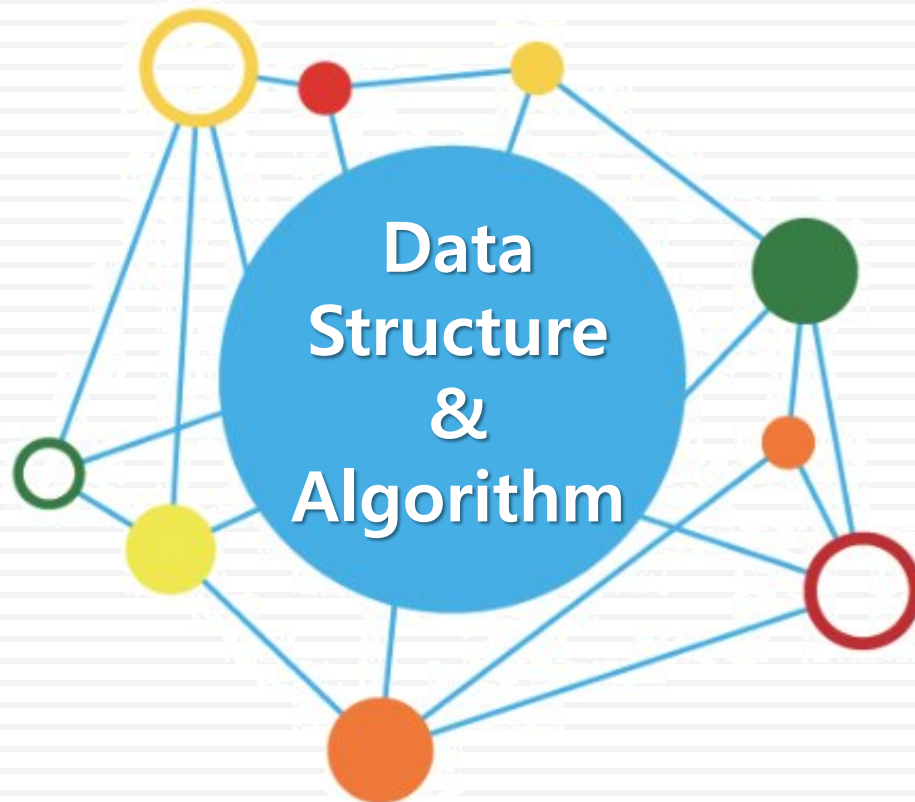


데이터 구조와 알고리즘

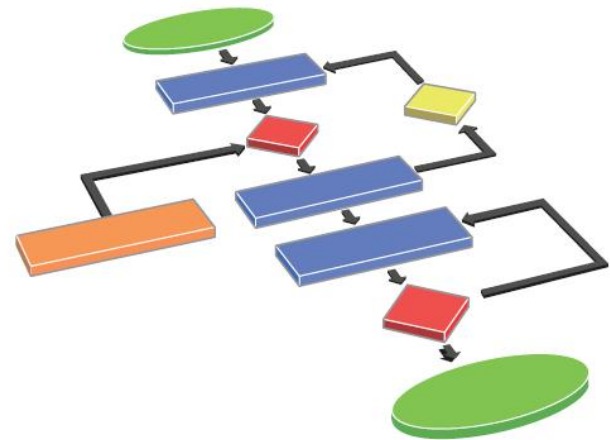
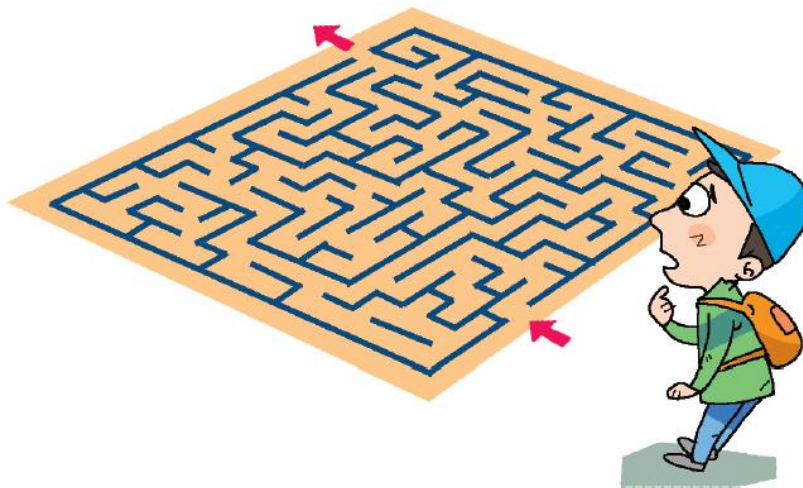


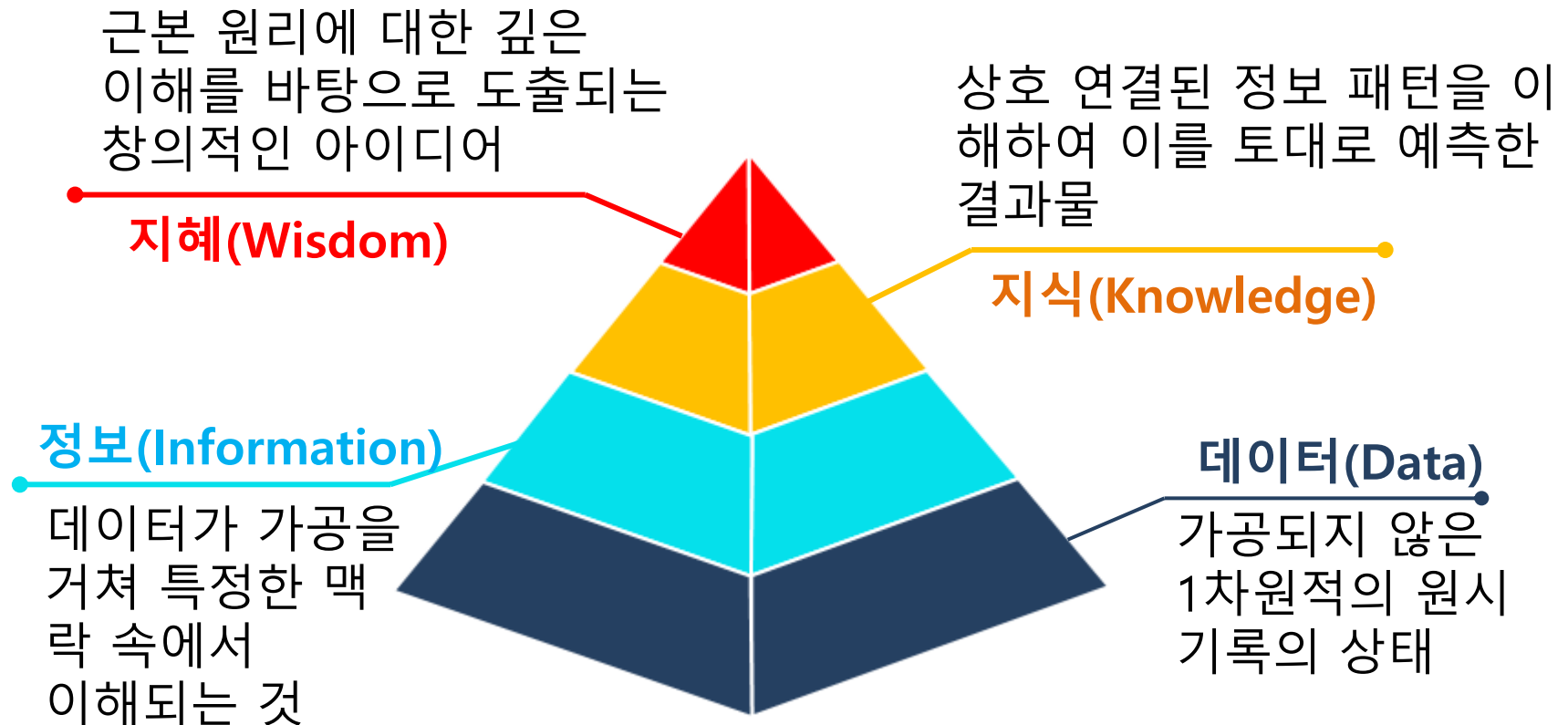
유 길현

❖ 데이터 구조(Data Structure)

- 컴퓨터 과학에서 데이터에 대해 효율적인 접근 및 수정을 가능케 하는 조직, 관리, 저장을 의미한다.
- 데이터 구조는 **데이터 값의 모임**, 또 **데이터 간의 관계**, **데이터에 적용할 수 있는 함수나 명령을 의미**한다.
- 데이터 구조의 선택 문제는 **효율적인 알고리즘**을 사용할 수 있게 할 수 있고 효과적으로 설계된 데이터 구조는 실행시간 혹은 메모리 용량과 같은 **자원을 최소한으로 사용**하면서 **연산을 수행**하도록 해준다.

- ❖ **알고리즘**(라틴어, 독일어: Algorithmus, 영어: Algorithm)
 - **문제를 해결하기 위한 절차나 방법**을 의미한다.
 - 어떠한 행동을 하기 위해 만들어진 명령들의 **유한 집합** 이다.
 - 컴퓨터 프로그램은 정교한 알고리즘들의 집합이라 할 수 있고 수학이나 컴퓨터 과학에서 말하는 알고리즘은, 보통 반복되는 **문제를 풀기 위한 작은 프로시저(진행절차)를 의미**한다





❖ 프로그래밍 언어

- 사람의 언어 $\leftrightarrow$ 기계의 언어
- 언어의 종류별로 고유의 문법이 있다
- 컴퓨터에게 동작을 지시할 수 있는 **명령어(데이터, 동작) 세트**와 그들의 **규칙(문법)**으로 구성
- 코딩(Coding)은 컴퓨터에서 동작할 수 있도록 프로그래밍 언어로 작성하는 작업

❖ 프로그래밍 언어 기술

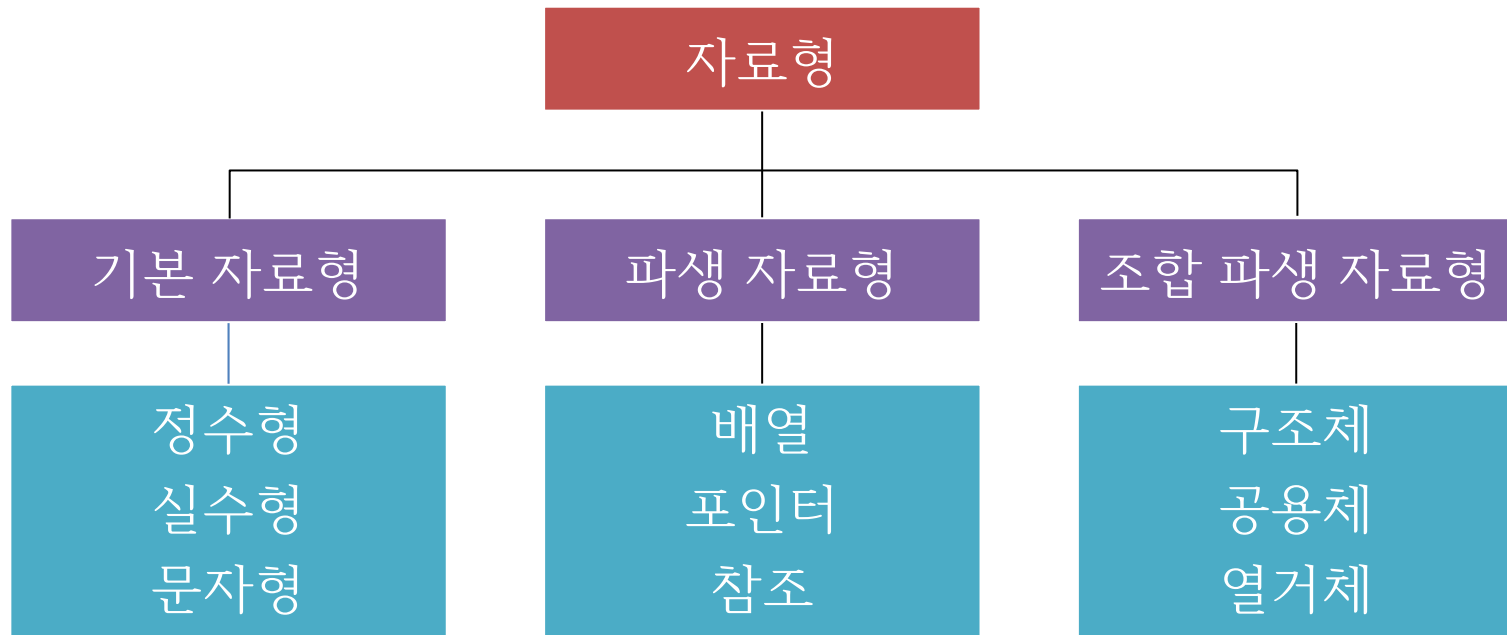
- 겉모양 ———> **문법구조(Syntax)**
- 속내용 ———> **의미 구조(Semantics)**

❖ 데이터와 연산

- 컴퓨터는 모든 데이터를 숫자로 취급하며 연산을 통해 새로운 정보로 가공
- 데이터의 구조 설계
 - 무슨 데이터를 다루는가?
 - 데이터를 어떤 구조로 관리할 것인가?
 - 데이터를 어떻게 가공할 것인가?
 - 데이터를 어떻게 처리 하느냐에 따라 프로그램의 성능이 달라질 수 있다.

❖ 데이터와 연산

- C언어의 자료(데이터)형 – 데이터의 성격을 결정



❖ 데이터와 연산

■ 연산(operation)

- 다른 저장장소(메모리)로 옮기거나 값을 변경
- 사칙 연산, 대입연산, 논리연산, 비교 연산 등
- 입력된 데이터를 가공하여 새로운 데이터를 만들어 내는 작업

구 분	연산자	비고
산술 연산자	+, -, *, /, %	산술연산, %는 나머지 연산자
대입 연산자	=, +=, -=, *=, /=, %= &=, ^=, =, <<=, >>=	오른쪽 값을 왼쪽에 대입
관계 연산자	<, <=, >, >=, ==, !=	대, 소, 등가 관계
논리 연산자	&&, , !	AND, OR, NOT
증감 연산자	++, --	1증가, 1감소
비트 연산자	&, , ~, ^	AND, OR, NOT, XOR
시프트 연산자	<<, >>	비트 단위 시프트

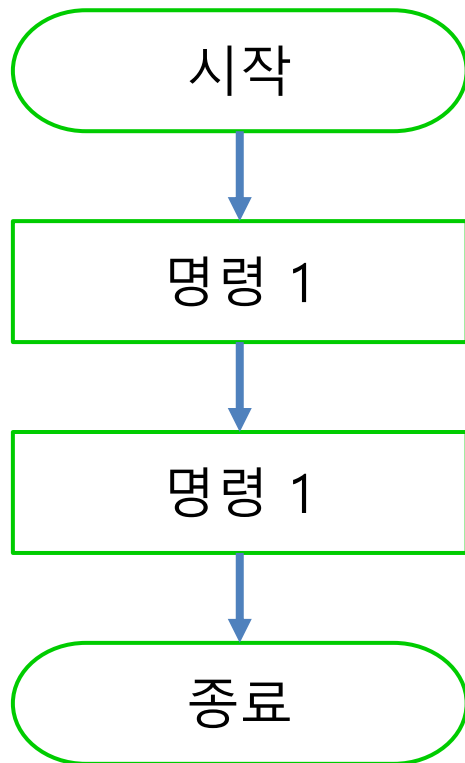
❖ 프로그램 문장

- 프로그램 작성에 있어서 기본적인 단위
- 프로그램 문장은 각각 세미콜론(;)으로 끝난다.
- 문장 구성 방법

변수 선언	자료형	이름	;
배열 선언	자료형	이름 [크기]	;
포인터 선언	자료형	* 이름	;
참조 선언	자료형	& 이름	;
구조체 선언	struct	이름 { 멤버 변수 ...; }	;
함수 선언	반환 자료형	이름 (매개 변수)	

❖ 순차적 동작

- 컴퓨터는 주어진 데이터를 대상으로 어떤 동작을 순차적으로 수행하도록 설계되어 있다.

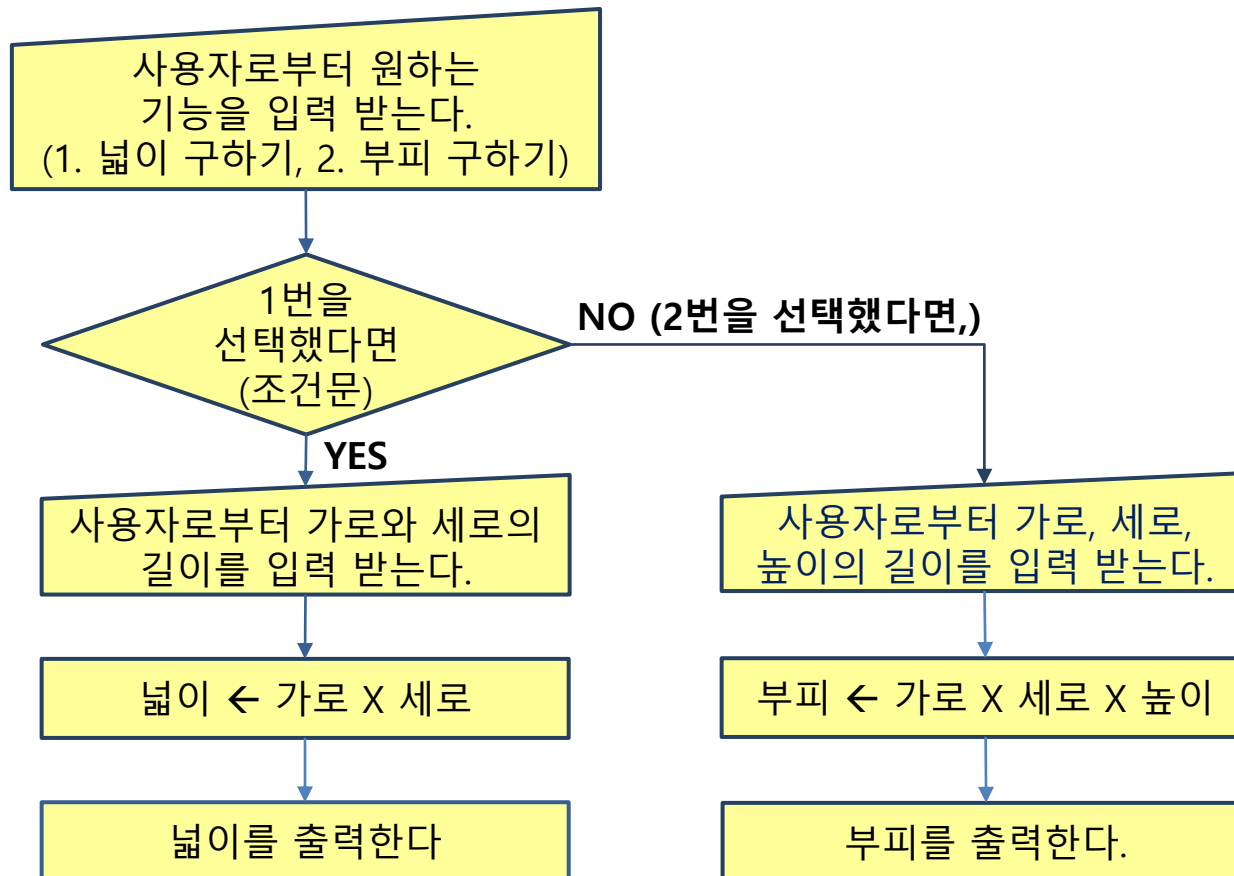


절차적으로 넓이를 구하게 방식

- ① 가로 길이 $\leftarrow 30$
- ② 세로 길이 $\leftarrow 20$
- ③ 넓이 $\leftarrow$ 가로 길이 $\times$ 세로 길이
- ④ 계산한 넓이를 화면에 출력

❖ 동작의 흐름을 바꾸는 제어문

- 특정 조건에 의해서 선택적으로 다른 작업을 수행하거나 어떤 구간을 반복적으로 작업

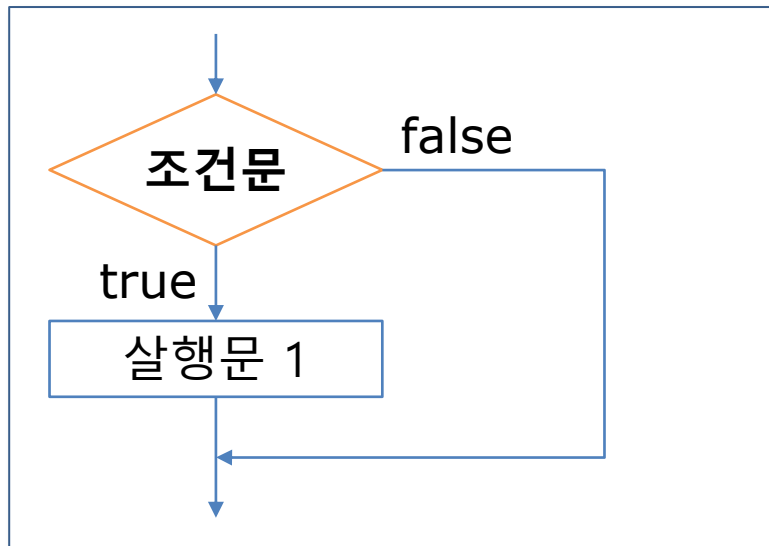


❖ 동작의 흐름을 바꾸는 제어문

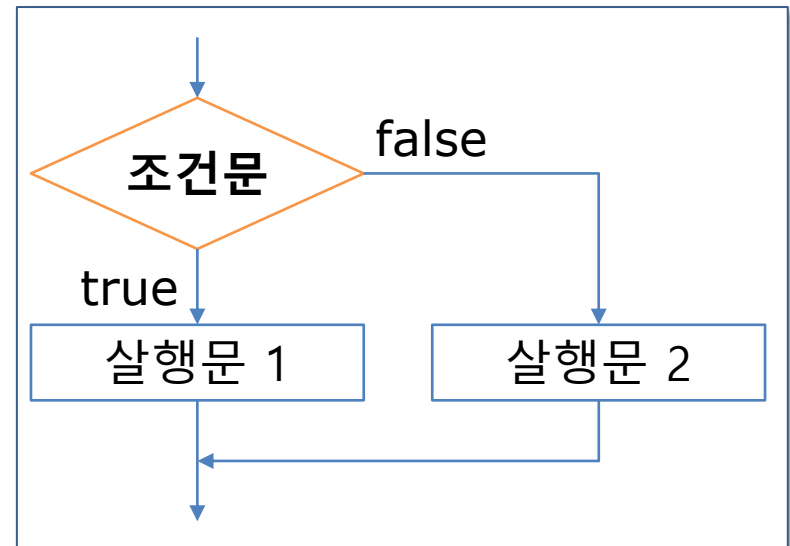
- 조건문 : 조건에 따라 선택된 동작으로 분기

조건 선택문	if	선택 사항의 개수가 적을 때 사용
	if ~ else if ~ else if	
	switch ~ case	선택 사항의 종류가 많을 때 사용

if 문



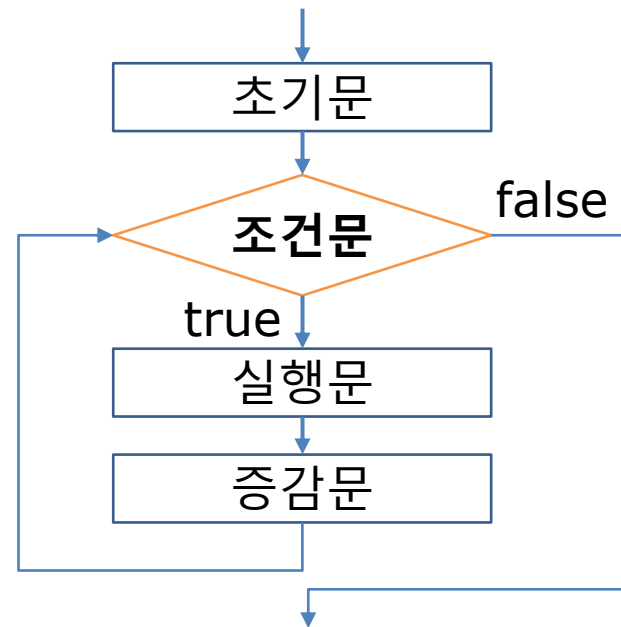
if ~ else 문



❖ 동작의 흐름을 바꾸는 제어문

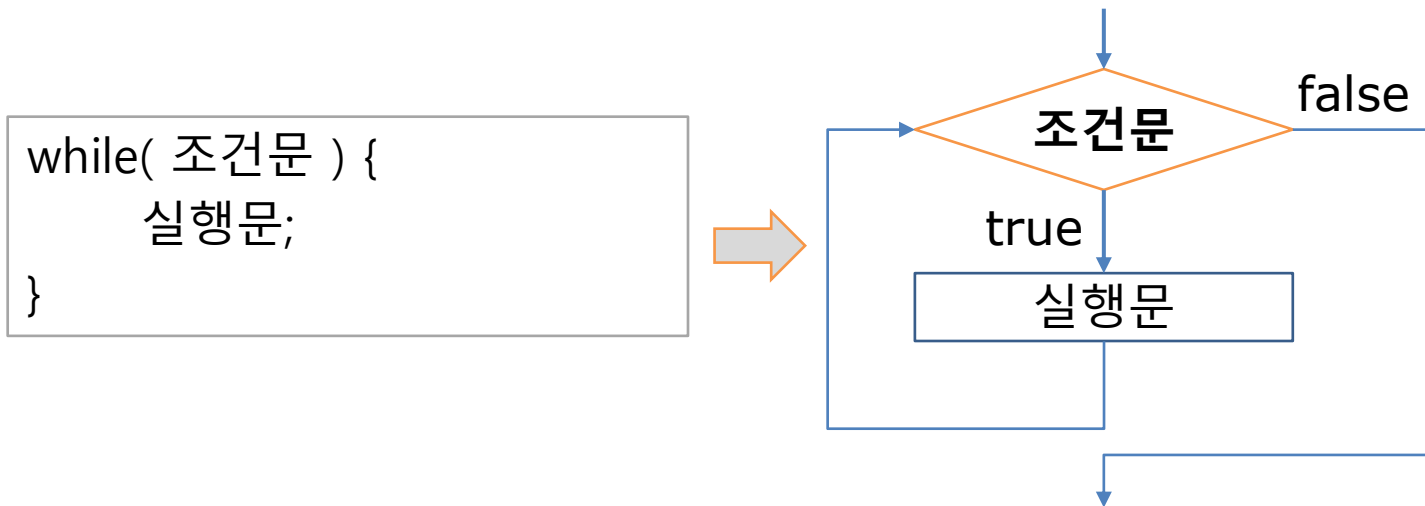
- 반복문 : 일련의 세트로 구성된 절차를 반복
 - **for** 문 : 지정된 횟수 만큼 반복해 수행

```
for(초기문; 조건문; 증감문){  
    실행문;  
}
```



❖ 동작의 흐름을 바꾸는 제어문

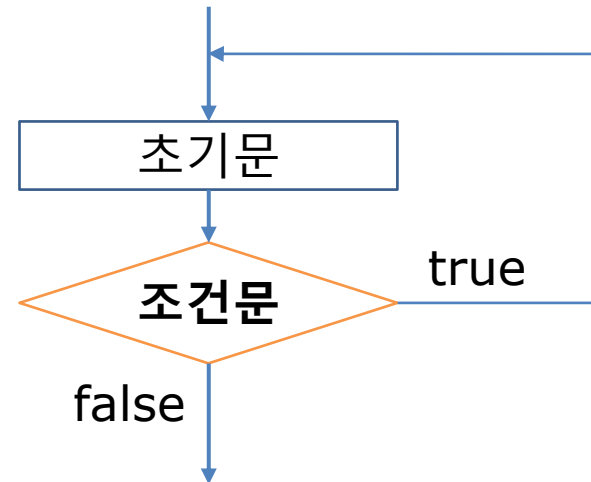
- 반복문 : 일련의 세트로 구성된 절차를 반복
 - **while** 문 : 조건 식을 먼저 검사하고, 조건이 만족되는 동안 반복 수행



❖ 동작의 흐름을 바꾸는 제어문

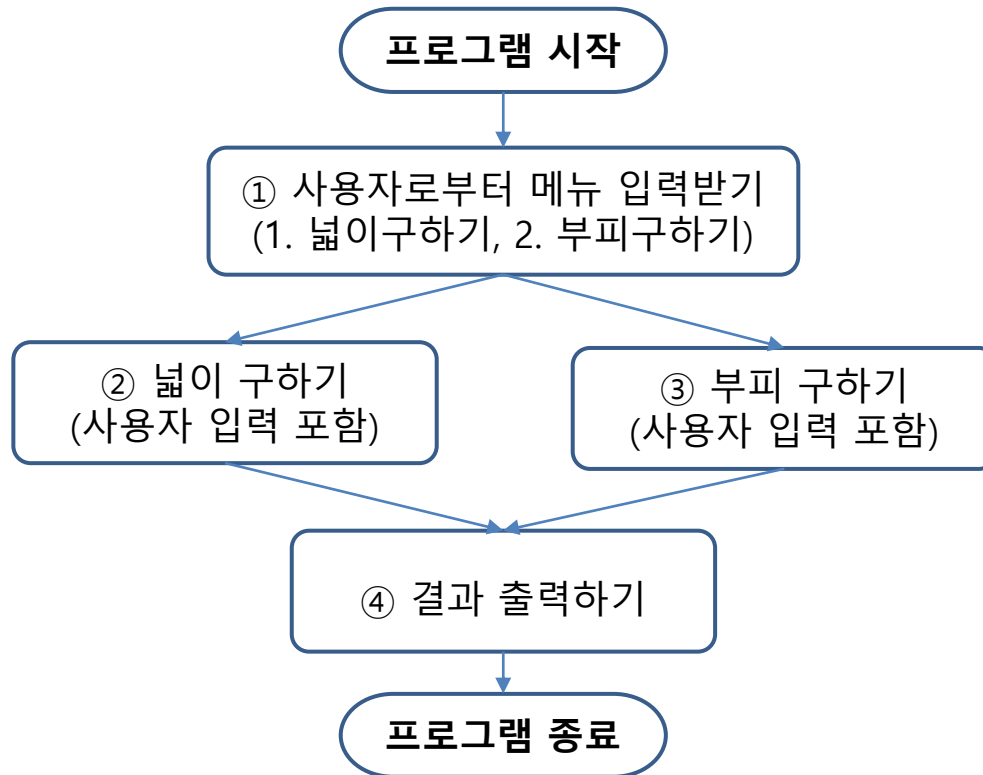
- 반복문 : 일련의 세트로 구성된 절차를 반복
 - **do ~ while** 문 : 수행을 먼저 하고 조건이 만족 되는 동안 반복 수행

```
do{  
    실행문;  
}while(조건문);
```



❖ 독립적 단위 기능을 수행하는 함수

- 복잡한 코드들을 독립적인 기능이나 데이터를 기준으로 모듈로 나누어 구성
- 자주 사용하는 기능을 함수로 구현해 필요시 호출 하여 사용



- ❖ 9개의 서로 다른 자연수가 주어질 때, 이들 중 최대값을 찾고 그 값의 위치를 찾는 프로그램 만들기
 - 서로 다른 9개의 자연수 3, 29, 39, 12, 57, 74, 40, 85, 61 이라면, 이 중 최대 값은 85이고, 이 값은 8번째 수이다.
 - **입력** ⇨ 첫 째 줄 부터 아홉째 줄까지 한 줄에 하나의 자연수를 입력한다. 단 0보다 크고 100보다 작은 자연수를 입력한다.
 - **처리** ⇨ 입력된 데이터를 비교하여 가장 큰 수와 위치를 찾는다.
 - **출력** ⇨ 첫 번째 줄에는 입력된 수를 입력 순서대로 출력하고, 둘 째 줄에는 가장 큰 수의 위치와 값을 출력한다.

❖ 입력 값의 최대값을 찾고 그 값의 위치를 찾는 프로그램

```
#include <stdio.h>

int main()
{
    int num[9]={0};
    int i, max, maxlocation;

    printf("배열에 원소 9개를 입력 하시오!!\n");
    for(i=0; i<9; i++)
    {
        printf("num[%d] = ", i);
        scanf("%d", &num[i]);
    }
    printf("\n");

    for(i=0; i<9; i++)
    {
        printf("%d ", num[i]);
    }
    printf("\n");
```

❖ 입력 값의 최대값을 찾고 그 값의 위치를 찾는 프로그램

```
max = num[0];
for(i=0; i<9; i++)
{
    if(max < num[i]){
        max = num[i];
        maxlocation = i+1;
    }
}

printf("\n 최대 값 : %d , 위치 : %d\n", max, maxlocation);

return 0;
}
```

❖ 프로그램 = 데이터 구조 + 알고리즘

- 예) 최대값 탐색 프로그램 = 배열 + 순차탐색

데이터 구조

score[]

80	70	90	...	30
----	----	----	-----	----

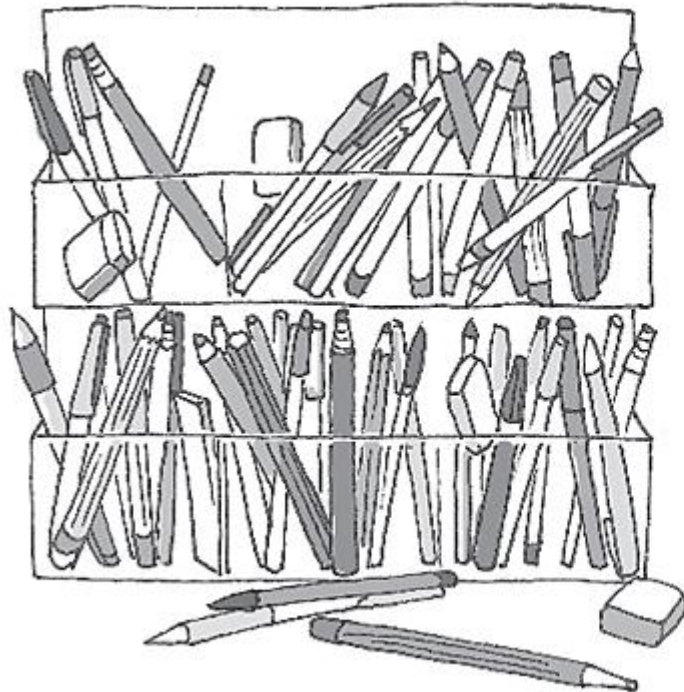
알고리즘

```
tmp ← score[0];  
for i ← 1 to n do  
    if score[i] > tmp  
        then tmp ← score[i];
```

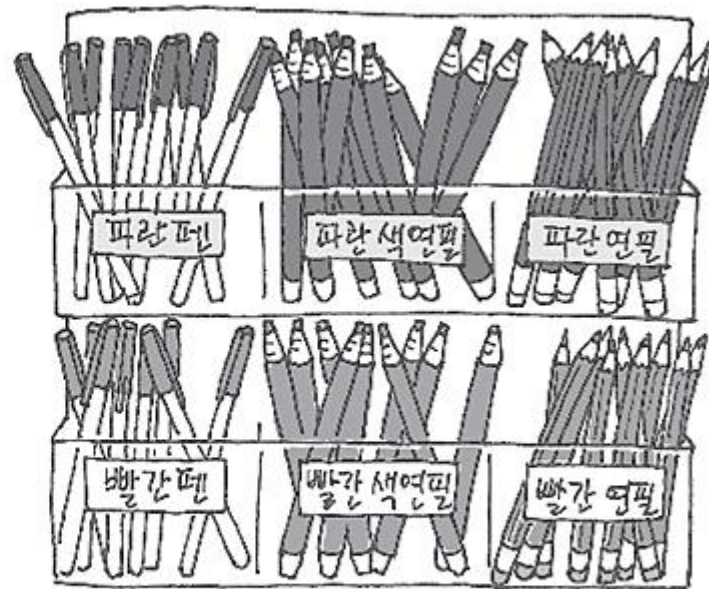
유사코드 표현

❖ 데이터 구조란

자료(Data)의 집합을 의미하며 각 원소들이 논리적으로 정의된 규칙에 의해 나열되며 자료에 대한 처리를 효율적으로 수행할 수 있도록 자료를 구분하여 표현한 것



(a) 자료구조를 적용하기 전



(b) 자료구조를 적용한 후

❖ 데이터 구조의 사용 목적

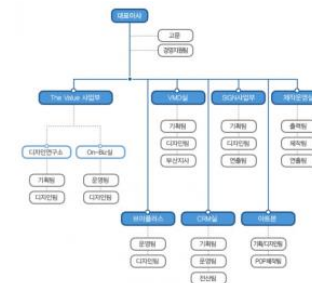
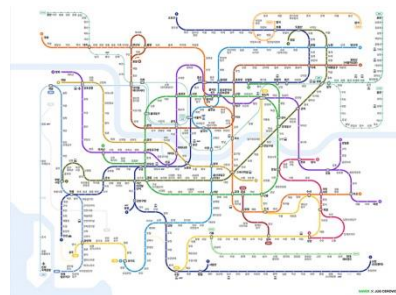
- 효율적인 프로그램을 작성할 때 고려사항은 **저장공간의 효율성과 프로그램 실행 시간의 신속성**이다.
- 프로그램에서 시용하기 위한 자료를 기억장치의 **공간 내에 저장하는 방법**과 저장된 그룹 내에 존재하는 **자료간의 관계, 처리 방법** 등을 정의하고 분석하는 것을 말한다.

❖ 데이터 구조의 특성

- 효율성 : 효율적인 데이터의 관리 및 사용
- 추상화 : 복잡한 자료, 모듈, 시스템 등으로 부터 핵심적인 개념만 간추려 내는 것
- 재 사용성 : 다양한 프로그램에서 동작할 수 있도록 범용성 있게 설계

❖ 일상 생활과 데이터 구조의 비교

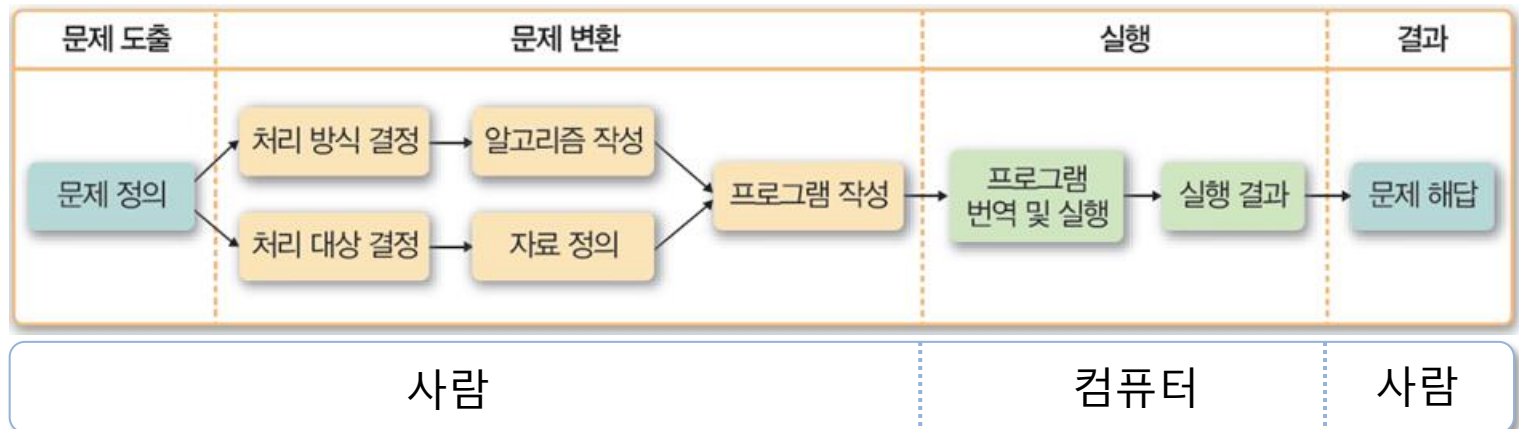
일상 생활의 예	데이터 구조
물건을 쌓아 두는 것	스택
영화관 매표소의 줄	큐
작업 리스트	리스트
영어 사전	사전 탐색구조
지도	그래프
조직도	트리



데이터 구조가 지니고 있는 의미

❖ 컴퓨터 분야에서 데이터 구조를 왜 배워야 하는가?

- 컴퓨터가 효율적으로 문제를 처리하기 위해서는 문제를 정의하고 분석하여 그에 대한 최적의 프로그램을 작성해야 한다.
- 데이터 구조는 데이터들이 일정한 원리에 의해 짜임새 있게 배열되어 있는 상태로 이 구조에 알맞은 알고리즘을 사용 하면 데이터 사용의 효율이 훨씬 높아진다.
- 문제 해결 단계



-
- 비트는 0 또는 1의 정보를 저장하는 정보의 최소단위이다.
- | | | | | | | | | | |
|-----|------|-----|-----|-----|-----|-----|-----|-----|----------------------|
| 비트 | bit | | | | | | | | |
| 니블 | bit | bit | bit | bit | | | | | |
| 바이트 | bit | bit | bit | bit | bit | bit | bit | bit | bit |
| 워드 | byte | | | | | | | | byte byte byte |
- 바이트는 8비트로 총 256가지의 정보를 저장할 수 있는 저장공간이다.
- 일반적으로 워드는 4바이트, 총 32비트로 구성된다.

❖ 십진수

- 우리 인간이 일상 생활에서 이용하는 가장 친숙한 수
- 10진수의 각 자리는 오른쪽부터 $1(10^0)$ 자리, $10(10^1)$ 자리, $100(10^2)$ 자리 순

$$\begin{aligned} \boxed{2} \quad \boxed{5} \quad \boxed{6} &= 2 \times 10^2 + 5 \times 10^1 + 6 \times 10^0 \\ &= 200 + 50 + 6 \end{aligned}$$

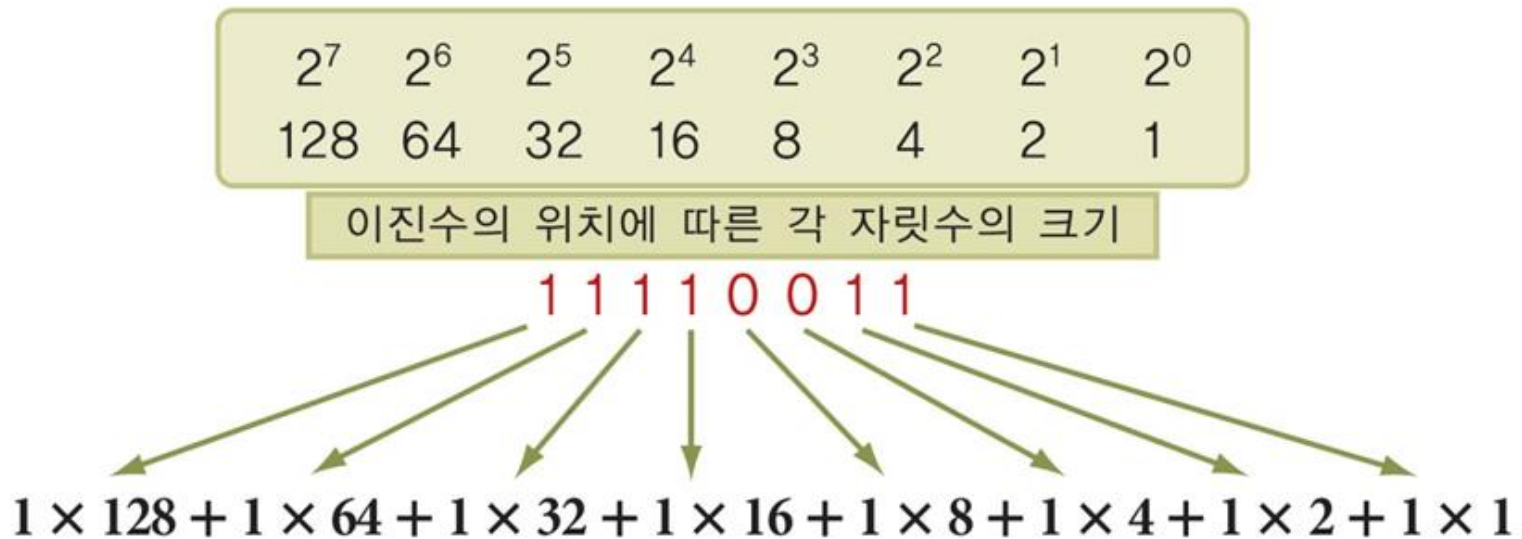
10^2 자리

10^1 자리

10^0 자리

❖ 2진수

- 0과 1의 두 가지 표현으로 각 자릿수를 표시
- 라이프니츠(Gottfried Wilhelm von Leibniz 1646~1716)
 - 독일의 철학자, 법학자이면서 수학자
 - 미적분과 함께 현대 대표 문명 기기인 컴퓨터가 탄생하는 데 결정적인 '0과 1'의 이분법을 개발



❖ 10진수를 이진수로 변환

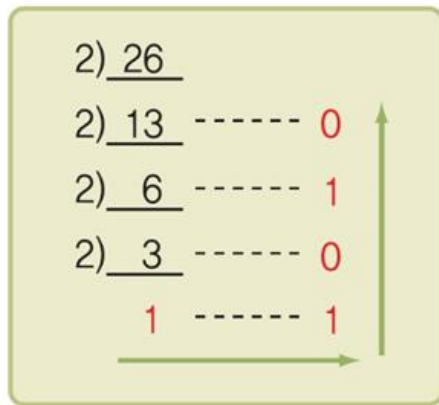
- $26 = 11010_2$

단계 1 : 주어진 값을 2로 나누고 그 나머지를 기록한다.

단계 2 : 몫이 0이 아니면 계속해서 새로운 몫을 2로 나누고 그 나머지는 기록한다.

단계 3 : 몫이 0이면 원래 값의 2진 표현은 나머지가 기록되는 순서대로 왼쪽에서 오른쪽으로 나열한다.

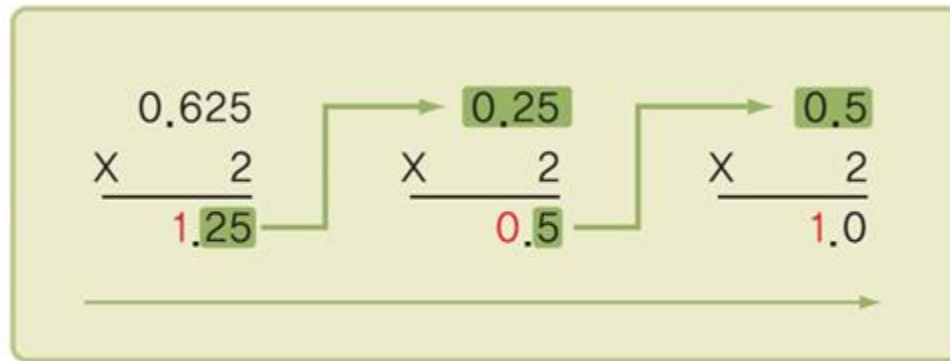
- 변환 과정



$$26 = 11010_2$$

❖ 10진수의 소수를 이진수로 변환

- 2를 곱해 나온 단위를 소수부분이 0이 될 때까지 나열
- $0.625 = 0.101_2$ 변환과정



$$0.625 = 0.101_2$$

❖ 2진수를 표현하는 세 가지 방법

표현 방법	특징
부호의 절대값 형식	• MSB값을 바꿔 음수를 간단히 표현 • +0(00000000)과 -0(10000000)의 존재로 논리적 문제 • n비트로 $-(2^{n-1}-1) \sim +(2^{n-1}-1)$의 범위를 표현
1의 보수 형식	• $(A-B)$뺄셈을 $(A+(B \text{의 } 1 \text{의 보수}))$로 변환하여 계산 • +0(00000000)과 -0(11111111)의 존재로 논리적 문제 • n비트로 $-(2^{n-1}-1) \sim +(2^{n-1}-1)$의 범위를 표현
2의 보수 형식	• $(A-B)$뺄셈을 $(A+(B \text{의 } 2 \text{의 보수}))$로 변환하여 계산 • 덧셈 연산에서 발생하는 오버플로우 처리가 1의 보수 형식보다 간단 • 컴퓨터 시스템에서 실제로 사용하는 형식 • n비트로 $-2^{n-1} \sim +(2^{n-1}-1)$의 범위를 표현

❖ 2진수의 실수 표현

- 고정 소수점 표현
 - 소수점이 항상 최상위 비트의 왼쪽 밖에 고정되어 있는 것으로 취급하는 방법
 - 고정 소수점 표현의 **00010101**은 **0.00010101**의 실수 값을 의미
- 부동 소수점 형식의 표현
 - 고정 소수점 형식에 비해서 표현 가능한 값의 범위가 넓음
 - 실수를 구분하여 표현

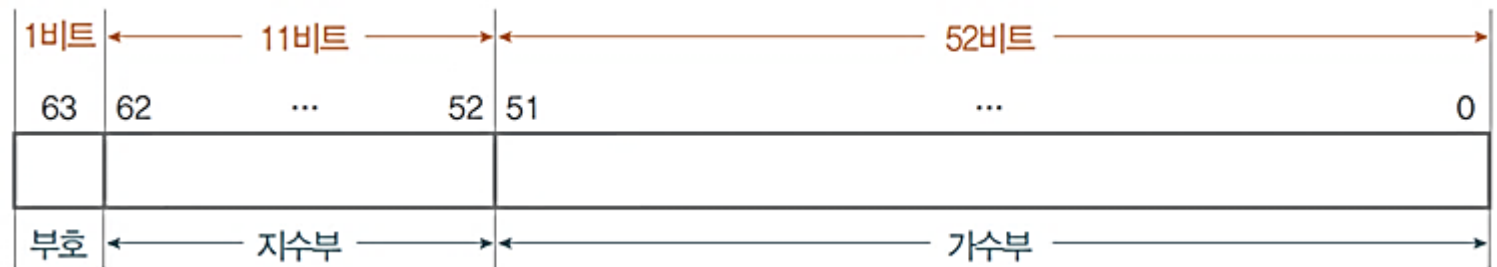
$$213 = \underbrace{0.213}_{\text{소수부}} \times \underbrace{10^3}_{\text{밑수 (base, radix)}} \longrightarrow \text{지수}$$

❖ 2진수의 실수 표현

- 부동 소수점 형식의 표현
 - 4바이트를 사용하는 부동 소수점 형식



- 8바이트를 사용하는 부동 소수점 형식

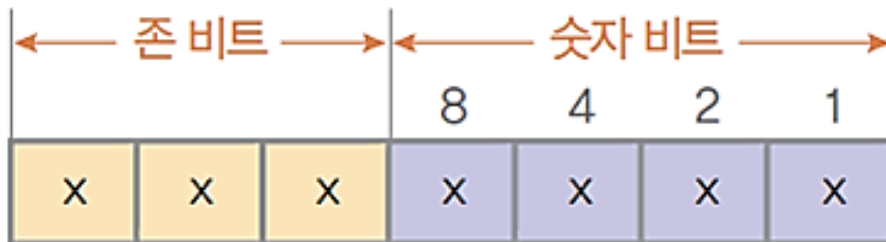


❖ 문자 데이터의 표현

- 문자에 대한 이진수 코드를 정의하여 사용
- 문자에 대한 이진수 코드 표준
 - BCD코드(Binary Coded Decimal)
 - EBCDIC코드(Extended Binary Coded Decimal Interchange Code)
 - ASCII코드(American Standard Code for Information Interchange)
- 유니코드(Unicode)

❖ ASCII 코드

- 7비트를 사용하여 문자 표현
 - 상위 3비트 : 존 비트
 - 하위 4비트 : 2진수 비트
 - 존 비트와 2진수 비트를 조합하여 10진수 0~9와 영어 대문자/소문자, 특수문자를 표현
- ASCII 코드의 구성



❖ ASCII 코드

■ 코드표

하위 \ 상위	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	END	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	W(\)	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

A가 있는 자리의 열 (존 비트) '100'과 행 (숫자비트)'0001'을 연결한 '1000001'이 A에 대한 ASCII 코드

※ 코드의 의미

GS	Group Separator	RS	Record Separator	US	Unit Separator
----	-----------------	----	------------------	----	----------------

❖ 유니코드

- 세계 각국의 언어를 통일된 방법으로 표현할 수 있도록 정의한 국제 표준 코드(ISO/IEC 10646)
- 2바이트(16비트)를 조합하여 하나의 글자를 표현하기 때문에 1바이트 코드로 표현할 수 없었던 다양한 언어를 표현.
- 한글 표현

항 목	조합형	완성형
표현 방법	• 자소에 의미 부여 • 조합할 수 있는 모든 글자 표현 (한글 11172자)	• 음절에 의미 부여 • 사용 빈도에 따라 추출된 한글 2350자만 지원
자소 구분	• 자소당 5비트 끊으면 가능	• 구분하기 위해서 먼저 조합형 코드로 변환해야 한다. (변환 테이블 이용)
한자 사용	• 현재 수용하고 있지 않음 (5천여자 수용 가능)	• 4884자를 수용하고 있음

❖ 논리자료

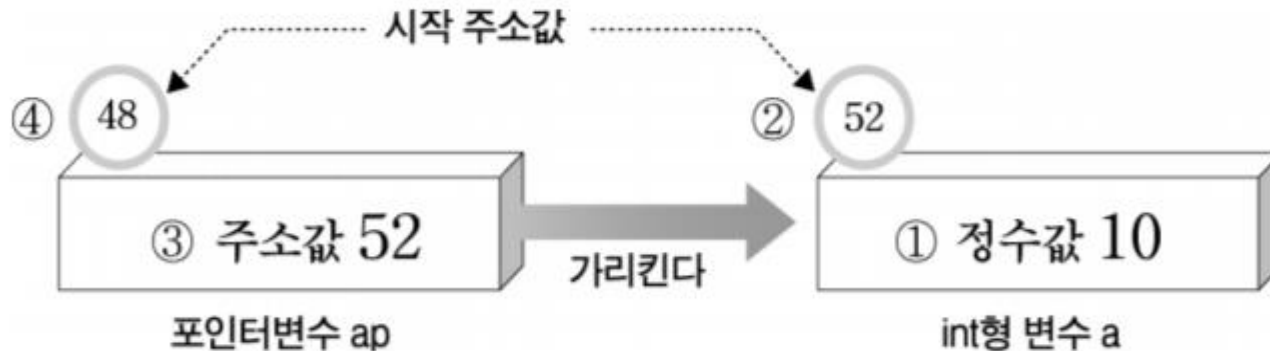
- 논리값을 표현하기 위한 자료 형식
 - 참(True)와 거짓(False), 1과 0
- 1바이트를 사용하여 논리자료를 표현하는 방법

구분	참	거짓
방법 1	최하위 비트를 1로 표시 00000001	전체 비트를 0으로 표시 00000000
방법 2	전체 비트를 1로 표시 11111111	전체 비트를 0으로 표시 00000000
방법 3	하나 이상의 비트를 1로 표시 00000001 or 00000100	전체 비트를 0으로 표시 00000000

❖ 포인터 데이터

- 메모리의 주소를 표현하기 위한 자료 형식
- 변수의 주소나 메모리의 특정 위치에 대한 주소를 저장하고 특정 변수의 포인터를 구하기 주소연산자(&)를 사용

```
int a = 10;           // int형 변수 선언, 정수 값 10으로 초기화  
int *ap = &a;         // int 포인터 변수 선언, a의 시작주소 값으로 초기화
```



❖ 문자열(String) 데이터

- 여러 문자로 이루어진 문자의 그룹을 하나의 자료로 취급하여 메모리에 연속적으로 저장하는 자료 형식
- 하나의 문자열 자료에 포함된 부분문자열을 표현하는 방법

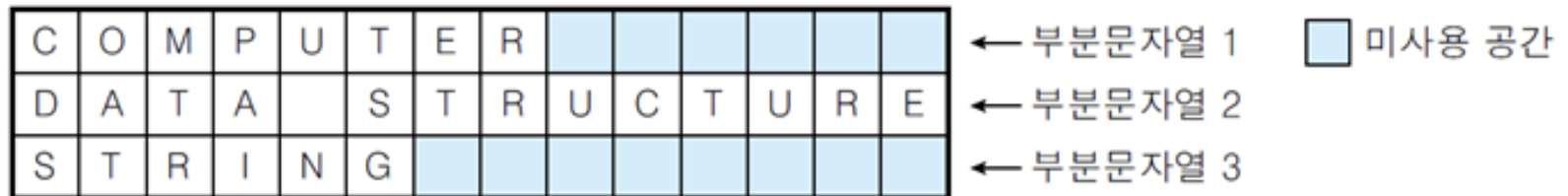
구분	부분 문자열 표현 방법
방법 1	부분문자열 사이에 구 분자를 두고 연속 저장
방법 2	가장 긴 부분문자열의 길이에 맞추어 고정 길이로 연속 저장
방법 3	부분문자열을 연속 저장하고 각 부분문자열에 대한 포인터를 사용

❖ 문자열(String) 데이터

- 방법 1. 구분자를 사용하는 표현

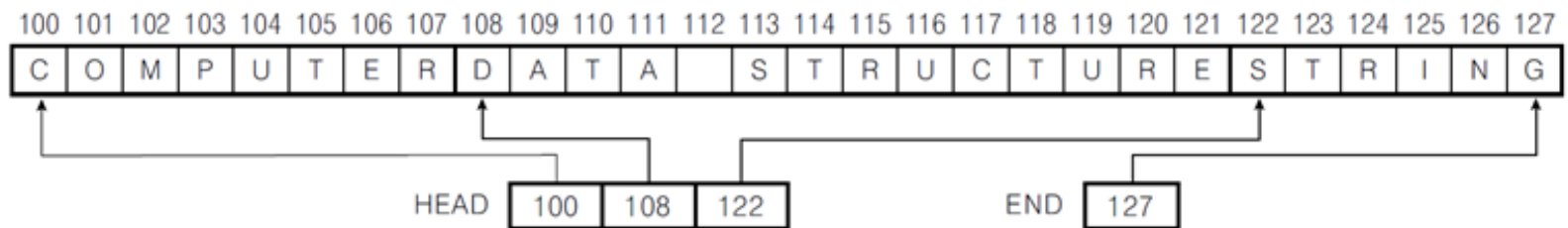


- 방법 2. 고정길이를 사용하는 표현



- 방법 3. 포인터를 사용하는 표현

메모리 주소



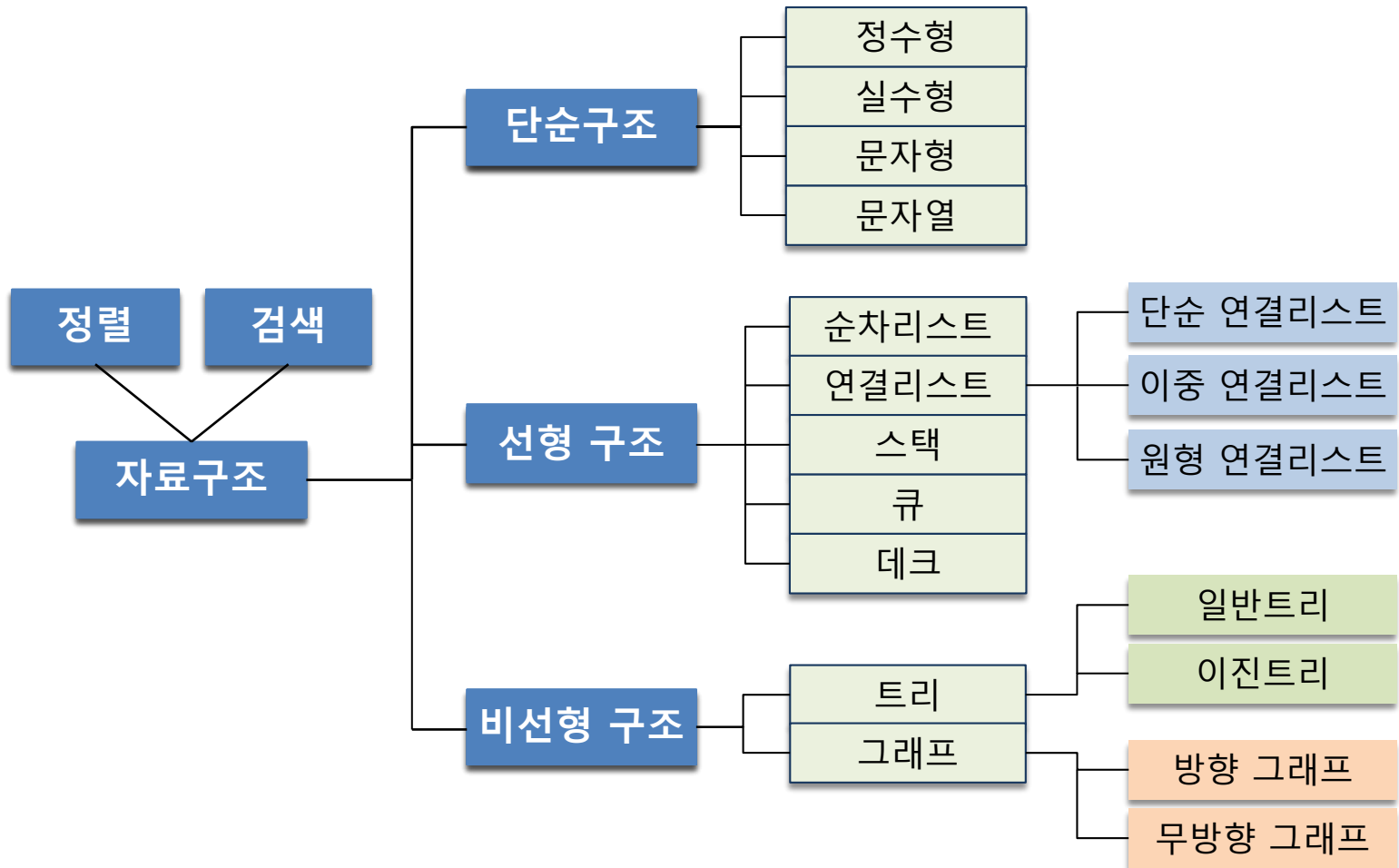
❖ 문자열(String) 데이터

■ 부분 문자열 표현 방법과 비교

비교 항목 \ 방법	메모리 이용률	부분 문자열 탐색 시간
구분자를 사용하는 방법	문자열 길이 + 구분자 길이 ⇒ 효율적	문자 비교 연산 시간 + 구분자 식별 시간 ⇒ 비효율적
고정 길이로 저장하는 방법	가장 긴 부분문자열 길이 × 부분문자열의 개수 ⇒ 비효율적	문자 비교 연산 시간 ⇒ 효율적
포인터를 사용하는 방법	문자열 길이 + 포인터 저장 공간 ⇒ 효율적	문자 비교 연산 시간 + 포인터 주소 연산 시간 ⇒ 효율적

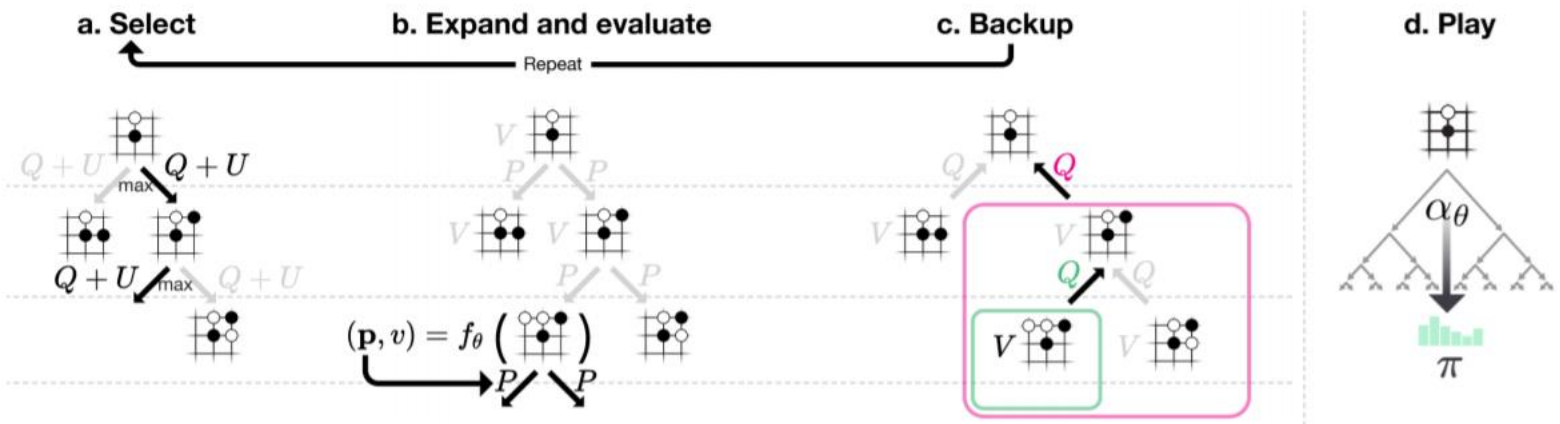
❖ 데이터 형태에 따른 분류

- 데이터 구조의 형태에 따른 분류



❖ 게임 트리 탐색 알고리즘

- 게임에서 장기, 바둑과 같이 두 플레이어가 번갈아 가면서 한번씩 게임을 하는 방식
- 알파고 인공지능 바둑 프로그램은 250^{150} 의 경우의 수를 모두 탐색하지 않고, 제한된 시간 안에 가장 승리할 가능성이 높은 경로를 탐색
- 탐색의 전략이 인공지능 바둑 프로그램의 성능을 좌우함



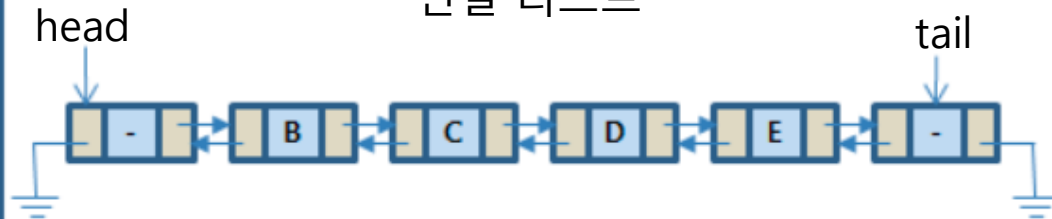
몬테카를로 트리 탐색(Monte-Carlo tree search)

선형 리스트

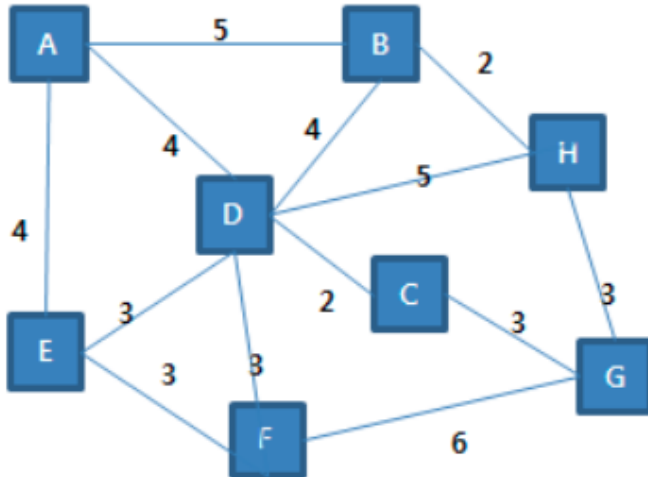


배열

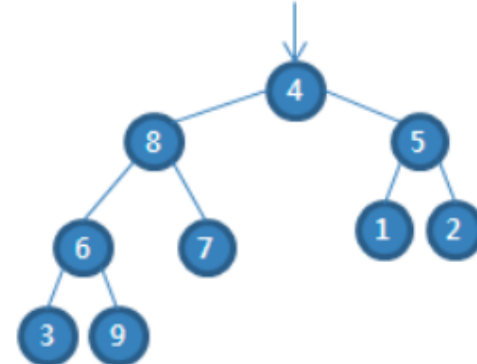
연결 리스트



그래프



트리
root



❖ 데이터의 표현, 프로그래밍 기법

- 데이터구조와 알고리즘 이해를 위한 기본지식
- 컴퓨터에서의 자료 표현 방법과 알고리즘 이해, 프로그래밍 기법

❖ 순차 데이터구조, 연결 데이터 구조

- 데이터의 논리적 순서와 저장되는 물리적 순서를 일치시키는 순차자료구조의 구성 방법
- 물리적 순서를 고려하지 않고 논리적인 순서대로 연결하여 데이터를 구성하는 연결 자료구조의 구성 방법
- 순차데이터구조와 연결 데이터구조의 차이점

❖ 스택과 큐

- 스택과 큐 데이터구조의 특징 및 연산 방법
- 순차데이터 구조와 연결데이터 구조를 이용한 스택과 큐의 구현
- 스택과 큐의 차이점

❖ 트리, 그래프

- 트리와 그래프의 개념
- 1 : n의 계층형으로 구조화한 트리 및 m:n 관계를 구조화한 그래프에 대해 알아보고 그 특징과 연산 방법
- 트리, 그래프를 배열을 이용한 데이터 자료구조 표현 방법과 포인터를 이용한 연결 데이터구조 표현방법으로 구현하는 방법

❖ 정렬과 검색

- 데이터구조를 이용한 정렬과 검색
- 선택·버블·퀵·삽입·셸·병합·히프·트리 정렬의 알고리즘을 살펴보고 프로그램으로 구현하는 방법
- 순차·이진 검색, 이진 트리 검색과 해싱 등의 방법

강의방법 안내 및 평가



이론 강의

1시간

- 교재의 내용 이론적 설명
- 데이터 구조에 대한 종류
- 데이터구조 프로그램의 기본 논리 및 구성 순서



프로그래밍 실습

3 시간

- C프로그래밍 언어 사용환경에서 실습
- 프로그램코드 작성 시 발생하는 문제 해결
- 설계된 논리의 구현 및 검사

평가 항목	평가 점수
출 결	20점, 4회 이상 결석 시 F(Fail)로 처리됨
중 간	20점, 필기 시험
기 말	20점, 필기 시험
과 제	40점, 과제제출 4회
기 타	총점 60점 이하는 F(Fail)로 처리됨

평가 방법
상대평가

IT CookBook, C로 배우는 쉬운 자료구조(개정3판)

